コンピュータサイエンス第一 第四回課題解答解説

今回のポイント

- 再帰関数(関数が自分自身を呼び出す)
- 再帰の２つの部品:「終わりの条件(基底)」と「小さい問題への分解」
- 文字列のスライス `s[1:]`、`s[:-1]`、`s[1:-1]` と、添字 `s[0]`、`s[-1]`
- `isalpha()` / `isascii()` / `lower()` による文字の判定と正規化

各問の解答解説

問 1

```python
def fibonacci(n):
    if n < 0:
        return None
    if n < 2:
        return n
    return fibonacci(n - 2) + fibonacci(n - 1)
```

これによって、`n` 番目のフィボナッチ数が返ってきます。例えば `fibonacci(10)` は **55**、`fibonacci(0)` は **0**、`fibonacci(1)` は **1** です。プログラムの流れとしては、まず `n < 2` のとき、つまり `n` が０か１のときは定義どおり $F_0 = 0$、$F_1 = 1$ なので、`n` をそのまま返します。ここが再帰の「終わりの条件」です。それ以外のときは、定義 $F_n = F_{n-2} + F_{n-1}$ をそのままコードにして、`fibonacci(n - 2) + fibonacci(n - 1)` を返します。

細かい解説

`fibonacci(3)` がどう計算されるか追ってみましょう。

```
fibonacci(3)
 = fibonacci(1) + fibonacci(2)
 = 1 + (fibonacci(0) + fibonacci(1))
 = 1 + (0 + 1)
 = 2
```

呼び出すたびに引数が小さくなり、必ず０か１に行き着くので、再帰が止まります。終わりの条件がないと、関数が自分を呼び続けて `RecursionError` になります。`n < 0` の場合は、フィボナッチ数が定義されていないので `None` を返すようにしました。これがないと `fibonacci(-1)`

> は `n - 2`、`n - 1` と負の方向に進み続けて止まりません。テスト表の「None or error」に対応しています。この書き方は `fibonacci(30)` で約 270 万回も関数を呼び出す、実用的ではない方法です(問題文にもあるとおり、練習のための実装です)。それでもこの課題の範囲なら数秒以内に終わります。

注. この問題では `while` や `for` などの繰り返し命令は使えません。繰り返しはすべて再帰で表現します。

問 2

```
def is_palindrome(s):
    if len(s) == 0:
        return True
    if not (s[0].isalpha() and s[0].isascii()):
        return is_palindrome(s[1:])
    if not (s[-1].isalpha() and s[-1].isascii()):
        return is_palindrome(s[:-1])
    return s[0].lower() == s[-1].lower() and is_palindrome(s[1:-1])
```

これによって、アルファベットだけを見て大文字小文字を無視したときに回文なら **True**、そうでなければ **False** が返ってきます。例えば `is_palindrome("Able was I ere I saw Elba")` は **True**、`is_palindrome("a bc a")` は **False** です。プログラムの流れとしては、まず文字列が空(`len(s) == 0`)なら、調べる文字が残っていないので回文とみなして `True` を返します。これが終わりの条件です。次に、先頭 `s[0]` がアルファベットでなければ、空白や記号は無視する決まりなので、先頭を取り除いた `s[1:]` で判定し直します。同様に、末尾 `s[-1]` がアルファベットでなければ、末尾を取り除いた `s[:-1]` で判定し直します。先頭も末尾もアルファベットのときは、`lower()` で小文字にそろえて比べます。等しければ、両端を取り除いた `s[1:-1]` が回文かどうかを再帰で調べます。等しくなければ `False` です。

細かい解説

`is_palindrome("Ab, a")` の流れを追ってみましょう。

```
"Ab, a"   先頭'A' 末尾'a' → 同じ (小文字で比較)  → "b, " を調べる
"b, "     末尾' ' は文字でない → 末尾を削る       → "b," を調べる
"b,"      末尾',' は文字でない → 末尾を削る       → "b" を調べる
"b"       先頭も末尾も'b' → 同じ → "" を調べる
""        空なので True
```

> `and` は左側が `False` ならその時点で全体が `False` と決まり、右側を実行しません。そのため、先頭と末尾が違うときは無駄な再帰をせずにすぐ `False` が返ります。長さ1の文字列(例えば`"a"`)では、先頭と末尾が同じ文字です。比べると等しく、`s[1:-1]` は空文字列になるので `True` が返ります。長さ1を別に場合分けする必要はありません。`s[1:-1]` は、長さ1や2の文字列でもエラーにならず、空文字列になります(スライスは範囲外でもエラーにならないためです)。`"a b"` は空白を無視すると`"ab"` なので `False`、`"madam i'm adam"` は記号と空白を無視すると `madamimadam` なので `True` になります。

注. `isalpha()` だけだと、ひらがなど英字以外も True になります。問題文のヒントにあるとおり、`isascii()` と組み合わせて英字だけを対象にします。`isalpha() & isascii()` のように `&` を使っても動きますが、この解答では普通の論理演算の `and` を使っています。